RAI: Safe Payments Through Committee Changes Without Stopping

Anonymous Author(s)
Regular paper

***Abstract*—Nano and FastPay confirm each account's payments separately, with no global order, in one or two rounds of votes. Two things remain hard. When the validators change, old votes can still complete a confirmation after the handover, so a careless handover can confirm two conflicting payments. And when an account signs two conflicting payments, by mistake or on purpose, most systems leave it stuck. RAI adds epochs. Each epoch ends in a checkpoint built from signed reports of the outgoing validators, and the next epoch votes while the previous one closes, under a few rules that keep the two from contradicting each other. Checkpoints also resolve every double spend, so its owner gets the account back. We prove safety and termination and model-check the handover rules. Across six regions, a prototype keeps confirming through transitions that stall a stop-and-drain handover, with latency tails ten times shorter.**

***Index Terms*—Byzantine fault tolerance, reconfiguration, asset transfer, consensusless payments, model checking**

I. Introduction

A payment moves money from one account to another. If every account has a single owner, there is no need to put all payments of the world in one sequence: the owner orders the payments of their own account, and the validators only have to make sure that no two incompatible versions of that account's history both become final. Guerraoui et al. showed that this makes asset transfer strictly easier than consensus [1], and systems such as FastPay [2], Astro [3], Zef [4], and Nano [5] use it to confirm a payment in one or two rounds of votes, with no leader and no block interval.

Such systems leave two problems open, and the two are connected.

Changing the validators. Validators join, leave, and move weight. In Nano the voting weight of a validator is whatever the accounts delegating to it hold right now, read by every node from its own ledger, so two honest nodes can count the same vote with different weight and can disagree on who is in the quorum. FastPay fixes the set of validators for the lifetime of the system and lists reconfiguration as future work, because there is "no natural point" at which all authorities switch [2]. The textbook remedy is to stop: end voting, wait until every pending confirmation has settled, agree on a snapshot, and start the new validators from it. Every transition then pauses the system for as long as the slowest confirmation takes.

Accounts that contradict themselves. An owner who signs two conflicting payments for the same position of their account, by a wallet bug or on purpose, splits the validators' votes. If neither side reaches the quorum, no payment at that position can ever be confirmed. In FastPay such an account "may become inaccessible" [2]; in Nano a fork whose final votes split stays unresolved, and every later payment of that account is blocked behind it (we measure this in Section X). A double spend should cost the attacker, not the honest owner who made a mistake, and it should not make the rest of the system slower: the owner should be able to recover the account once they stop contradicting themselves.

RAI. This paper presents RAI, a set of rules that gives an account-local payment system epochs, checkpoints, and committee changes without stopping, and that resolves every fork, so the owner of a forked account gets it back. Time is cut into epochs. In each epoch a fixed committee of validators votes on payments exactly as before. At the end of an epoch every outgoing validator freezes one signed *report* of what it saw; the outgoing committee agrees on one *checkpoint* built from these reports by a fixed recipe; and the next committee, known two epochs ahead, installs the checkpoint without voting on it. The checkpoint settles what the old epoch could still have confirmed and keeps at most one side of every unresolved fork, with a note of what may still happen to it. From then on every validator votes only the kept side, which becomes final within two epochs of normal operation, and the owner continues from it.

The hard part is the overlap. RAI does not stop at an epoch boundary: validators start voting in epoch $e + 1$ while epoch e is still closing. Votes already given in epoch e can still complete a confirmation after the boundary, and a checkpoint that ignored such a *latent* confirmation could let the next epoch confirm a conflicting payment. Four rules keep the overlap safe (Section VII): new-epoch confirmations wait for the checkpoint unless they carry proof that the old epoch cannot contradict them; outgoing validators keep adding that proof for a short while; votes cast before their signer knows the checkpoint do not count toward the fast path; and a validator does not switch sides across the boundary. Each rule is needed: TLA$^+$ model checking finds a violation within a few steps when any one of them is weakened, and none with all of them in place (Section IX).

Contributions. (1) A protocol for pipelined epoch transitions in account-local payment systems: immutable signed reports, a deterministic checkpoint recipe with *locks* that record what may still become final and which epoch they protect, lagged committees, and the four overlap rules. (2) A proof sketch of safety and termination relative to two abstract services, set reconstruction and agreement, whose interfaces RAI fixes

but whose mechanics it leaves open; full proofs are in the supplement. (3) Model checking of the handover rules, which shows each rule necessary by a counterexample without it and checks the complete rule set, including the rule that guards a whole account prefix. (4) A prototype in the RsNano implementation of Nano, evaluated across six regions against Nano and against a stop-and-drain transition built from the same code, with forks and with a Byzantine validator that equivocates in every phase, and on one host under a committee change, validator restarts, and a crashing departing validator.

RAI is for authenticated committees with an externally maintained fault bound; it does not address permissionless membership, and its liveness promise to an owner who keeps contradicting themselves is only that their account cannot hurt anyone else.

II. The Problem, by Example

Six validators $V_1, \ldots, V_6$ confirm payments; at most one of them, say V_6, is faulty. A block is confirmed when four validators have given it their *final vote*. Alice signs block A at position 7 of her account, paying Bob, and publishes it.

What a committee change must preserve. Suppose the committee is about to change, and the validators agree on a snapshot of all accounts to hand over. Validators V_1 to V_4 have each signed a final vote for A, but the network is slow: no single party has seen all four votes, and in particular the validators building the snapshot have seen at most three. If the snapshot leaves position 7 open, the new committee may confirm a conflicting block A' there. Later the four old votes meet, and A is confirmed too, by a perfectly valid certificate of the old committee. Two conflicting payments are final. Nothing in the agreement on the snapshot went wrong; the snapshot was simply built from a view of the votes that was not the only possible one. We call A *latent-final*: its certificate exists in the released signatures, though nobody has assembled it. The same holds for a certificate that outgoing validators who have not yet stopped may still complete after the snapshot.

The textbook remedy is to stop: end voting, wait until every certificate has either been assembled or become impossible, then snapshot. The wait is as long as the slowest confirmation, at every transition, and during it no payment is confirmed. Section X measures what this costs: under such a pause, blocks published during a transition wait for it.

RAI's answer is that the snapshot does not need to know whether A is final. It needs to know whether A *could* be final, and that is visible in the reports: a correct validator final-votes only a block it has seen notarized, and says so in its report. Four final voters and five reporters out of six overlap in at least three validators, at least two of them correct, so the snapshot sees A notarized in at least two reports and keeps it. The snapshot does not confirm A (it cannot know), but it *locks* position 7 for A, so the new committee cannot confirm A', and A's latent certificate stays compatible with everything that follows.

What a committee change should not cost. Nano has no epochs; a representative's weight changes whenever a delegating account moves, read by each node from its own ledger. Two correct nodes can thus see different quorums. Stopping is not an option there either, which is one reason the problem has stayed open. RAI wants both: committee changes with a clear, agreed membership and weight per epoch, and no stop.

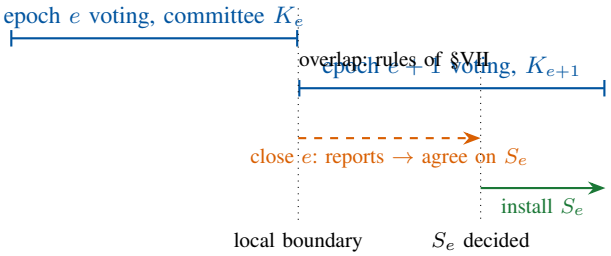


Fig. 1. A pipelined transition. Epoch $e+1$ voting starts at each validator's local boundary while epoch e closes. Most new blocks finalize during the overlap under the overlap rule; the rest finalize when S_e is installed.

Accounts that contradict themselves. Now suppose Alice signs two blocks at position 7, A paying Bob and A' paying Carol, and the validators split three against three. No block reaches four. In Nano the position is stuck for good, and so is every later block of Alice's. A deliberate attacker loses access to their own money, which is fine; an honest user with a buggy wallet loses it too, which is not. In RAI the next checkpoint keeps one of the two blocks, under a lock, and every validator that installs it votes only that block at position 7. So the kept payment becomes final within two epochs of normal operation, whatever Alice or a faulty validator does, and Alice's account continues from it. A deliberate attacker gets at most one of their two payments through. The next sections make these statements precise.

III. RAI at a Glance

Figure 1 shows one epoch transition. Three committees are involved: K_e votes on payments in epoch e and, at the end, decides the checkpoint S_e; K_{e+1} is already known (it was fixed by S_{e-1}) and votes in epoch $e+1$ while K_e is still closing; S_e fixes K_{e+2}. Boundaries are local: each validator ends epoch e on its own clock, freezes its report, and opens $e+1$ at once. Nothing requires the validators to stop together.

RAI is a set of rules. We state them here informally, with the thresholds of the six-validator example in parentheses, and spend the next sections explaining why each is needed.

Inside an epoch (Section V).

R1 A validator casts at most one *first vote* per account position per epoch, for the first valid block it sees there, and never switches.

R2 A block with q (4) first votes is *notarized*. A validator casts a *final vote* only for the block it first-voted, once it holds that block's notarization. A block is *final* with q (4) final votes, or on the fast path with $N-p$ (5) first votes from validators that know the previous checkpoint.

At the boundary (Section VI).

R3 At its local boundary a validator stops voting in epoch e, signs one immutable *report* naming what it holds final or notarized and what else it first-voted, and opens epoch $e+1$.

R4 The outgoing committee agrees on a checkpoint built from $N-f$ (5) reports by a fixed recipe: keep what any report shows final; *lock* a block that a report shows notarized; from the highest position down, *lock for recovery* a block r (3) reporters first-voted after installing the previous checkpoint, and keep a side where nothing is kept; promote a notarized block that is the only one kept at its position; carry every older lock forward unless evidence of its own origin epoch releases it.

R5 The next committee installs the decided checkpoint without voting on it. From then on a validator votes at a kept position only the kept block, and builds on a block kept only for recovery once it is notarized again.

While the previous epoch closes (Section VII).

R6 *Gate.* A block of epoch $e+1$ does not become final before S_e is installed, except under R7.

R7 *Overlap rule.* It may become final early if it, and every unresolved block below it, has q (4) supporters in epoch e and q (4) first votes in epoch $e+1$, and if every older recovery lock it bypasses is released by a witness of that lock's own origin.

R8 *Late notarization.* A validator that has left epoch e but not yet installed S_e also signs an epoch-e *support* for each block it first-votes in $e+1$, unless it supported a rival at that position in e. These supports count only toward the q of R7, never toward finality or reports.

R9 *Early votes.* A first vote cast before its signer installed S_e counts toward notarization and toward R7, never toward the fast path. An early final vote is released only with the witnesses of R7 in hand.

R10 *No switching across the boundary.* Until S_e is installed, a validator does not first-vote in $e+1$ a block that conflicts with one it supported at that position in e, nor any block where it has seen two or a checkpoint keeps one.

Two services are left abstract: how a validator obtains the sets behind another's report (*set reconstruction*) and how the outgoing committee picks one eligible checkpoint (*agreement*). RAI only states what it needs from them (Section IV).

IV. Model and Assumptions

Participants. Clients own accounts and sign blocks. A committee has $N = 3f + 2p + 1$ validators of equal weight, of which at most f are Byzantine; p is the number of responses the fast path may do without, not an extra fault allowance. Corruption is static: a Byzantine identity stays Byzantine, a correct one stays correct, including after it leaves. Who may join a committee, and that no committee exceeds f faults, are deployment assumptions. Our running example and prototype use $f = p = 1$, $N = 6$; Table I lists the thresholds that follow.

The numbers are chosen so that any two groups of q validators share at least $f+1$ members, hence at least one correct one ($2q - N = f + 1$); so that any q final voters and any $N-f$ reporters share at least $p+1$ correct ones ($q - 2f = p + 1$); and so that any $N-p$ fast voters and any $N-f$ reporters share at least r correct ones ($N-p-2f = r$). These three overlaps carry every safety argument of the paper.

TABLE I
Vote thresholds, and their values for $f = p = 1$, $N = 6$.

What	Needs	Example
Block notarized (also: exclusion witness)	$q = N - f - p$ first votes or supports	4
Block final, normal path	q final votes	4
Block final, fast path	$N - p$ settled first votes	5
Recovery lock in a checkpoint	$r = f + p + 1$ reporter first votes	3
Reports in a checkpoint	$N - f$	5

Accounts and blocks. Each account is a chain of owner-signed blocks starting at a genesis. A block names its parent and sits at the parent's position plus one; positions are heights in the account, not times. Two blocks of one account *conflict* if neither is an ancestor of the other. A block carries payments: a *send* debits its own account; a *receive* credits the account and names a send that must already be final. An owner *equivocates* by signing conflicting blocks. Safety covers equivocating owners; liveness promises them nothing while they keep contradicting themselves.

Network and cryptography. Signatures cannot be forged and hashes do not collide. Every vote binds its epoch, committee, account, position, kind, block, and the evidence under which it was cast, so a vote cannot be replayed in another context. Correct validators write what they sign to disk before sending it, keep blocks, votes, and evidence, and keep serving them after they leave an epoch. Safety needs no bound on message delay. Termination needs the usual: finite work, fair retry, and recurring windows in which messages arrive within a known delay and clocks drift by a bounded factor.

What a checkpoint contains. A checkpoint S_e is the agreed state at the end of epoch e: for each account the finalized prefix, the unresolved blocks the epoch kept, and *locks* on those blocks; plus balances and the identifiers of received sends. A lock says "keep this block, it may still be final" and records its *strength* (notarized, or recovery) and its *origin*: the epoch whose possible confirmation it protects. A lock is released only by evidence from its own origin epoch (Section VI). The committee of epoch $e+2$ is a deterministic function of S_e, so it is known throughout the transition from $e+1$ to $e+2$.

Two abstract services. RAI commits to sets by their hash and to candidates by their digest, and uses two services to turn these commitments into shared state. *Set reconstruction* lets any validator obtain the two sets a reporter committed to in its signed report; RAI needs that a reconstructed set matches the signed hash (integrity) and that a correct reporter's sets are eventually reconstructible at every correct validator

(termination). *Agreement* lets the outgoing committee decide one candidate checkpoint among those RAI declares *eligible*; RAI needs that all correct validators decide the same value, that the decided value is eligible, that the decision proof is verifiable by anyone who knows the committee and cannot be replayed, and that a decision is reached whenever $N - f$ correct members hold an eligible candidate. Any set-reconciliation mechanism and any Byzantine agreement protocol with external validity [6] fit; our prototype uses rateless set reconciliation [7] and a Kudzu-style election [8]. The supplement states the interfaces formally (C1–C2, A1–A4).

V. Inside an Epoch

Within an epoch RAI votes like Nano or FastPay, with two twists that pay off at the boundary: a validator never switches sides, and a final vote is preceded by a notarization.

Confirming a block. Alice signs block A at position 7 of her account and sends it to the six validators. Each validator checks the block against Alice's chain as it knows it (parent, balance, no duplicate receive) and, if it has not voted at position 7 in this epoch, casts a first vote for A (R1). With four first votes A is *notarized*; a validator that holds the notarization and first-voted A casts its final vote. Four final votes make A final. If five first votes arrive from validators that have installed the previous checkpoint, A is final at once, in one round trip; Nano always needs a second round (Section X).

Why the thresholds are safe. Any two groups of four validators out of six share at least two, and at most one of those is Byzantine. A correct validator supports one block per position (R1), so two conflicting blocks cannot both reach four supporters. Four supporters for A therefore *exclude* every rival: no rival can be notarized or final in this epoch. We call a set of q supporters an *exclusion witness*; it is the one piece of evidence the boundary rules will ask for again and again. Five first votes for A leave at most one correct validator free to support a rival, so a fast-final block has no rival with three honest supporters either; this is what makes the recovery threshold $r = 3$ of R4 work.

A double spend. Suppose Alice also signs A' at position 7, paying someone else, and shows A to three validators and A' to the other three (Figure 2). Neither reaches four. In Nano this position is stuck: final votes cannot be withdrawn, no block at position 7 will ever confirm, and every later block of Alice's account waits behind it. In RAI the next checkpoint decides what may still happen at position 7 (R4). It is built from five of the six reports, which show three first votes for one side and two for the other, say three for A. A is kept under a *recovery lock* of origin e and A' is dropped: from now on no validator admits A' or any other rival of A at position 7, so A' can never become final. Had the reports shown three against three, the block with the smaller hash would be kept.

How the fork ends. The checkpoint decides the position (R5). A validator that has installed it votes only A at position 7, and a validator that has seen both A and A' casts no vote

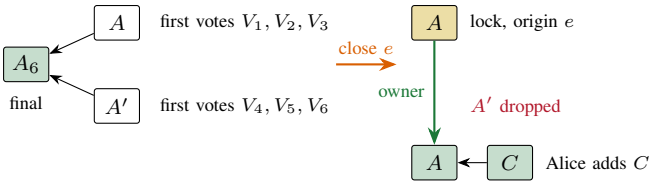


Fig. 2. A double spend at position 7. Neither block reaches four first votes. The five selected reports show three votes for A and two for A': the checkpoint locks A for recovery and drops A', which can never become final. In the next epochs every validator votes A, which becomes final; Alice may also add C on top of A.

there before it has the checkpoint (R10). So A collects four first and four final votes and becomes final by the second epoch of normal operation after the fork, whatever Alice or the Byzantine validator does (Theorem 3), and Alice's account continues from it. She need not act; to pay sooner she can sign a block C on top of A, admitted once A has four first votes again in the new epoch, and finalizing C finalizes A (Figure 2). A block on top of A' is refused.

Why the checkpoint must decide. Suppose validators could still vote A' whenever they held four epoch-e supporters for it, the witness that releases A's lock; that takes a Byzantine validator that supported both sides. Then in every epoch some correct validators vote A and some A', the Byzantine validator withholds the one vote either side needs, and the position stays open forever, as in Nano (supplement). With the checkpoint deciding, the witness still lets a validator accept a certificate for A' that already exists, so nothing final is contradicted, but it earns A' no new vote.

Durable votes. A correct validator writes its first vote and its final vote to disk before releasing them, so that a crash and restart cannot make it vote twice. Our prototype syncs these writes; Section X reports what that costs.

VI. Closing an Epoch

An epoch closes in three steps: every outgoing validator freezes a report, the outgoing committee agrees on a checkpoint built from $N-f$ reports, and the next committee installs it. Figure 3 follows Alice's account through one close.

Reports (R3). When a validator's local clock says the epoch is over, it stops first- and final-voting in it, and signs one header committing to two sets. The first is a tagged view of its ledger: every block it holds, tagged *final* (by an inherited checkpoint or by a certificate it has seen), *notarized* (four first votes seen, no finality), or *inherited* (kept by the previous checkpoint under a recovery lock, nothing new seen). The second set lists every block the validator first-voted in this epoch that the first set does not already show as notarized or final. The two sets are frozen: later events change the validator's live view, never its report. The header carries only the hashes of the two sets; whoever needs the sets reconstructs them (Section IV). A report is *usable* at a validator once both sets are reconstructed and every claim in them is backed by evidence that validator holds: four first votes behind every

report V_1	A_6: F	A: N	B: voted
report V_2	A_6: F	A: N	B: voted
report V_3	A_6: F	A: N	B': voted
report V_4	A_6: F	A: voted	B: voted
report V_6	A_6: F	A': N	

checkpoint S_e: A_6 final (inherited); A *locked, notarized* (V_1, V_2, V_3 show it notarized; V_6's claim for A' has no four first votes behind it and is dropped);
B *locked for recovery, origin* e (three reporter first votes, no rival with three); B' dropped.
Promotion: A is the only block kept at position 7 and has an epoch-e notarization $\Rightarrow$ A final in S_e.

Fig. 3. Five of six reports close epoch e for Alice's account. F = final, N = notarized, voted = first-voted but not notarized (the report's second set).

"notarized", a certificate behind every "final", the reporter's own signed first vote behind every entry of the second set. A Byzantine reporter can make its own report unusable, and nobody else's.

The second set has a subtle requirement. A reporter that first-voted a block the previous checkpoint already keeps under a recovery lock tags it *inherited* in the first set. If the second set excluded every block of the first set, this fresh vote would appear nowhere, and a latent fast certificate could vanish from the checkpoint (Section IX). The second set therefore excludes only blocks tagged notarized or final.

The recipe (R4). A candidate checkpoint names exactly $N - f$ usable reports from distinct members, in a fixed order, plus a manifest of the evidence behind their claims. From these inputs every validator computes the same state by the following recipe, which is deterministic and never consults the candidate's own certificate.

1) *Keep finality.* Every block a selected report shows final, with its ancestors and the sends its receives depend on, is final.
2) *Lock notarized blocks.* At each position, if a selected report shows a block notarized, backed by four first votes, keep it under a *notarization lock*. Two conflicting blocks cannot both be notarized (Section V), so there is at most one.
3) *Lock for recovery.* Count only *settled* first votes, cast after installing S_{e-1}. From the highest position down, where no block is notarized or kept by a lock placed above, lock a block with r (3) of them under a *recovery lock* of origin e. Then, from the lowest up, where nothing is kept, keep the previous checkpoint's block if eligible, else the one with the most, ties by hash. A block that may still become fast-final has r, so it or a child of it is locked; keeping a side even below three lets the next epoch converge.
4) *Carry old locks.* Every lock of the previous checkpoint is carried forward, unless a final block excludes its block, or, for a recovery lock of origin o, the manifest contains an exclusion witness of epoch o for a conflicting block. Nothing else releases a lock; in particular a newer epoch number never does.
5) *Promote.* For each account, starting at its last final block, walk up while the next block is the only one kept at its position and has an epoch-e notarization of first votes; those blocks become final with the checkpoint. Promotion never resolves a fork and never promotes a block kept by a recovery lock alone.

The thresholds make this safe (Theorem 1): whatever the old epoch could still confirm is kept as final or under a lock, and a lock can only be released by evidence of its own origin epoch, which cannot exist if the block it protects can still be final.

Why locks have an origin. A recovery lock says "three validators first-voted this block in epoch o; a fast certificate with five epoch-o votes may exist." Such a certificate is excluded exactly by four epoch-o supporters for a rival, and by nothing else: four epoch-$(o+1)$ votes for the rival say nothing about epoch o. So the lock records o, and a block may carry locks of several origins, one per epoch in which it was voted but not resolved. Releasing a lock on the strength of a newer epoch's votes would let two validators finalize different blocks (Section IX).

Eligibility, agreement, and installation (R5). A candidate is *eligible* if its $N - f$ reports are usable, its manifest hash matches, and the recipe yields the state whose digest it names. The outgoing committee runs the agreement service with this predicate; the service may be any Byzantine agreement protocol with external validity. The decided value is small, $N - f$ report headers and two digests, whatever the number of accounts. Since each header carries its reporter's signature, a validator that holds another version of a report, which only a Byzantine reporter can sign, authenticates the selected one from the value and reconstructs it. The next committee verifies the decision proof, reconstructs the inputs, recomputes the state, and installs it: its final set is the union of its base and what it had already finalized, replay applies each operation once, and nothing is rolled back. There is no second vote; the successor only checks.

After the checkpoint: the checkpoint decides. A lock guarantees that no rival of the kept block is admitted at its position until evidence of the lock's own origin releases it; a notarization lock is never released. The checkpoint also decides how validators vote there (R5): a kept block gets an instance at its own position in the open epoch, and a validator that has installed the checkpoint first- and final-votes only that block there, and above it only on top of it, even if it holds a witness that would release the lock. The witness still lets it accept a rival's certificate that already exists (Lemma 8), so this only withholds votes. A validator that has seen two blocks at a position also waits for the checkpoint before voting there (R10). Together with a recipe that always keeps a side, this makes the kept block final within two epochs of normal operation, with or without the owner (Theorem 3). The owner may also extend the kept block with a fresh child: at once under a notarization lock, and under a recovery lock once the

block is notarized again in the open epoch. Letting any kept tip be extended at once would be unsafe: a dropped side with an old witness may become final in the new epoch while a child's certificate finalizes the kept side (supplement, Case 5). In our fork workloads, whose owners never extend a forked position, validators resolved every fork this way (Section X).

VII. Voting While the Previous Epoch Closes

A validator opens epoch $e+1$ the moment it leaves epoch e, typically a few seconds before S_e is decided. During this overlap two things could go wrong: the new epoch could confirm a block that the old epoch's latent votes contradict, and the checkpoint could later rule against something the new epoch already confirmed. Rules R6 to R10 prevent both. We explain them with Alice's account at position 8, right after A became final at position 7.

The gate (R6). The simplest safe behaviour is to wait: a block of epoch $e+1$ collects first and final votes as usual, but no validator treats it as final until it has installed S_e and re-checked the block against it. If Alice's block B at position 8 is published during the overlap, its certificate is held, and the moment S_e arrives the held certificate takes effect, or is discarded if S_e kept a conflicting block. This is correct but it is also the stop-and-drain cost in disguise: every block published during a close waits for the close to end.

The overlap rule (R7). A block can skip the gate if the old epoch provably cannot contradict it. The proof is an exclusion witness in the closing epoch: four supporters for B in epoch e. If four validators supported B at position 8 in epoch e, no rival at position 8 can be notarized or final in epoch e (Section V), so S_e cannot finalize a rival, cannot lock a rival for notarization, and any recovery lock S_e places on a rival has origin e and is released by exactly this witness. So a validator that holds the epoch-e witness for B, four epoch-$(e+1)$ first votes for B, and an epoch-$(e+1)$ final certificate for B may finalize B before S_e is known. The same check must hold for every unresolved block below B on Alice's chain, because finalizing B also finalizes its ancestors; and if B bypasses a recovery lock inherited from an older checkpoint, the validator must hold a witness of that lock's own origin epoch. The rule is exactly the check a validator would perform after installing S_e, so a validator that finalizes B before and one that re-checks after reach the same verdict (Lemma 8).

Late notarization (R8). A block published after the boundary has no epoch-e votes at all, so the overlap rule would never apply to it and it would wait at the gate. Late notarization fixes this. A validator that has left epoch e but not yet installed S_e, when it first-votes B in epoch $e+1$, also signs an epoch-e *support* for B, unless it supported a different block at position 8 in epoch e. These late supports count toward the exclusion witness and toward nothing else: no notarization, no final vote, no report entry rests on them, since reports are frozen before they are cast. Safety is the same argument as before: one validator supports one block per position in epoch e, by a first vote or by a late support.

Early votes (R9). A fast certificate must not rest on a vote only some validators can justify. Suppose B conflicts with a block B' that S_e keeps under a recovery lock, and V_3, V_4, V_5 first-vote B early, without the epoch-e witness for B. V_1 later installs S_e, obtains the witness from the Byzantine V_6, releases the lock, and first-votes B too: five first votes, a fast certificate that only V_1 can justify. The next checkpoint may select reports without V_1's and keep B', and a later epoch finalizes B' while B's certificate is still valid (supplement). The rule removes the dependency: a first vote names the checkpoint its signer had installed; *early* votes, cast before S_e, count toward notarization and witnesses but never toward a fast certificate, and an early final vote is released only by a validator that holds the overlap witnesses, which its report then carries to the next checkpoint. A block first-voted during a close therefore takes two rounds instead of one.

No switching (R10). A validator that supported B' at position 8 in epoch e does not first-vote a conflicting B in epoch $e+1$ until it has installed S_e. Otherwise it could contribute to an epoch-$(e+1)$ notarization of B while its epoch-e vote still helps B' toward a latent certificate. At a position where it has seen two blocks, or which a checkpoint keeps, it casts no first vote at all before installing S_e: re-voting its old side early would let the split repeat in every epoch.

After installation. When S_e arrives, each validator re-checks its provisional epoch-$(e+1)$ work against the finalized history, the frontier, and the locks of S_e. Held certificates take effect; work on a block that S_e excluded is discarded, without clearing the validator's recorded first vote. Every position with one kept block gets an instance at that position in the open epoch, and a rival is refused while any lock on the kept block stands. If a kept block does not finalize in epoch $e+1$ either, its first votes are exposed again in the next reports, and the next checkpoint keeps it again, with a lock of origin $e+1$ if three reporters voted it: a block may carry locks of several origins, each released only by a witness of its own epoch. From then on every validator that installed the checkpoint votes it there.

VIII. Why It Is Safe and Why It Terminates

This section gives the arguments in words; the supplement has the formal statements and full proofs, numbered as here. All arguments are relative to the two services: safety uses the integrity of set reconstruction and the agreement, eligibility, and verifiability of agreement; termination additionally uses their termination properties.

One epoch (Lemma 1, account exclusion). Two sets of q supporters share a correct validator, and a correct validator supports one block per position. So at most one block per position has an exclusion witness, and if a block can become final, by q final votes or by $N-p$ settled first votes, no rival has a witness; if it can become fast-final, every rival has fewer than r supporters.

Notarized ancestry (Lemma 2). A child is first-voted only on a parent that is final, under a notarization lock, or notarized in the child's epoch, never on one kept by a recovery lock

alone. So two conflicting blocks final in epoch e, even only as ancestors, meet at one position where Lemma 1 or the lock excludes one.

Reports expose votes (Lemma 4). A correct reporter's frozen report tags a block it final-voted as notarized or final, and lists every other block it first-voted in the second set, whatever the block inherited from the previous checkpoint. Late supports are never in a report. This is why the second set must list fresh votes for blocks the predecessor already kept: without them a fresh vote vanishes (Section IX).

Theorem 1 (Closure preserves latent and late finality): If an epoch-e final certificate for B can be formed from released votes, even one completed after the reports froze or after S_e was decided, then every eligible checkpoint S_e holds B: as final, under a notarization lock, or under a recovery lock of origin e. No admissible evidence releases that hold.

Why. Count overlaps. A normal certificate has q final voters; the checkpoint selects $N - f$ reports; the two share at least $p+1$ correct validators, each of which notarized B before final-voting and says so in its report, so the recipe keeps B final or locks it. A fast certificate has $N-p$ settled first voters; it shares at least r correct validators with the reports; if none of them tagged B notarized, each lists B in its second set, so B has r settled reporter votes while every rival has fewer (Lemma 1) and no rival is notarized. A block above B on a conflicting branch has no correct voter (Lemmas 1 and 2), so the recipe, working down, locks B or a child of B. If B bypasses an older lock, every correct signer of its certificate holds the releasing witness (Lemma 5), and the manifest carries it. A lock of origin e on B can only be released by an epoch-e witness for a rival, which Lemma 1 excludes. A B final only as an ancestor is held below its descendant, and Lemma 2 leaves no rival a witness to release it. The argument uses only the signers and the selected reporters, so it does not matter when the certificate is assembled.

Lemma 8 (Same verdict before and after installation): A validator that finalizes B under the overlap rule before installing S_e accepts B if and only if it would accept B by re-checking after installing S_e; and a later checkpoint verifies a report that claims B final.

Why. The overlap rule asks for exactly the evidence that re-checking consults: the epoch-e witness for B rules out a final or notarized rival in S_e and releases any origin-e lock on a rival; the matching-origin witnesses release any older lock; and S_{e-1} was already installed when the overlap began. Without matching-origin release two correct validators can reach different verdicts on the same evidence, depending on arrival order (Section IX).

Theorem 2 (Transfer safety across epochs): Finalized histories at correct validators are compatible and never rolled back, finalized balances are nonnegative, and each send is received at most once.

Why. By induction over epochs, jointly with Theorem 1, which each step uses for the closure before it. Inside an epoch Lemma 2 brings two conflicting final blocks to one position, where Lemma 1 or a notarization lock excludes them. At a boundary, agreement gives every correct validator the same eligible checkpoint, Theorem 1 says it preserves every formable old certificate, and promotion (R4) only finalizes a block with a closing-epoch notarization, which excludes rivals. For the overlap, the first unresolved position of an early-final prefix has an epoch-e witness, so no closing-epoch certificate or promotion can contradict it, and Lemma 8 makes all correct validators agree on which overlap blocks are final. The checkpoint's construction is deterministic in its inputs (Lemma 9), so correct validators compute the same state, and installing is a union of final sets with duplicate-suppressed replay, never a rollback.

Theorem 3 (Termination): Every started handover is eventually decided and installed by correct validators. An owner whose new blocks are uncontested sees them final and in a checkpoint eventually. A forked position whose parent is final becomes final, on one side, by the end of any two consecutive epochs of normal operation, whatever the owner and the Byzantine validators do; the owner then continues from it.

Why. A correct reporter's report is finite and backed by evidence it holds, so under set reconstruction and the data service it becomes usable at every correct validator; $N - f$ usable reports make an eligible candidate constructible, and agreement decides one; successors verify the proof and install. For accounts: in a window where $N-f \geq q$ correct validators are open, an uncontested block collects q first votes, hence a notarization and q final votes; a block caught by a boundary is preserved by Theorem 1, an omitted one is retried. At a fork, in a good epoch every correct validator votes; one that saw both blocks, or whose checkpoint keeps one, votes after installing the predecessor and only the block it keeps. If the position is still open at the end, every correct validator has seen both blocks, the next checkpoint keeps a side, preferring the one it kept before, so the side changes at most once, and in the next good epoch all of them vote that side, which becomes final. The rules only withhold votes, apart from the recipe's choice, which keeps every block that may be fast-final (Theorem 1). Without them a Byzantine validator that supported both sides keeps the position open by withholding the one vote either side needs.

IX. Model Checking the Handover

Several rules of Sections VI and VII guard against interleavings that are easy to miss in a written argument. We wrote TLA$^+$ models of the handover and checked them with TLC, each with the full rule set and with switches that weaken one rule, to show that the rule is needed. Each takes one account with $N = 6$, $f = p = 1$, five correct validators and one Byzantine validator whose signatures are available for every block, vote kind, and epoch, and whose report is chosen by the adversary subject to the usability checks. Agreement is the nondeterministic choice of one eligible candidate; TLC explores every interleaving, with validators identified up to permutation. The models, configurations, and TLC outputs with the violating traces are in the supplement and the artifact.

Close three voting epochs and two checkpoints, no early voting; the recipe with locks of several origins and matching-origin release. Switch: a second set that omits fresh votes for blocks tagged inherited.

Overlap one closing and one open epoch; validators cross their boundary one by one, vote early, late-notarize, and finalize under the overlap rule or after installing. Switches turn off matching-origin release, late supports as evidence, and the separation of notarization from late supports.

Handoff early voting, two checkpoints, and a committee change with the Byzantine validator in both committees; the early-vote rule on or off; a releasing witness reaches the second checkpoint only through a selected correct report, which models a withheld signature.

Prefix two positions of one account: a parent with a rival at position 1 and two children at position 2, so that the prefix-wide guard of R7 is exercised; committees identical, disjoint, or sharing members. Switches check only the finalizing block instead of the prefix, and let a child inherit its parent's witness.

Recovery a child of each forked block at the next position, installation, and settled votes. Switch: any kept tip may be extended without a new notarization.

Resolve one forked position over three epochs and two checkpoints; a Byzantine validator that votes both sides or withholds; epochs flagged as good (synchronous) or not. Switch: the termination rules.

Recover four epochs, three checkpoints, two or three rivals with a child each, blocks replayed to anyone, an owner that forks or not. Switches: the recipe's vote count, keep rule, and order.

Table II summarizes; the supplement explains each violation. Extending any kept tip is unsafe even without early votes. A recipe that counts all first votes and works bottom-up loses a fast-final child, and so does a bottom-up recipe on settled votes with a forking owner; the recipe of Section VI passes the safety, side-change, and recovery checks by simulation, and safety in every run of up to 17 steps, two more than the bottom-up order needs to fail. Probes show that the invariants are not vacuous.

The models are small: one or two positions, two to four epochs, no receives, no restarts, $N \in \{4, 6\}$. Their abstractions only add behaviours, except the recover model's owner and Byzantine policies, so a violation in the model is a violation of the protocol, but a clean run is evidence, not proof.

TABLE II
TLC RESULTS. STATES ARE DISTINCT STATES OF COMPLETED RUNS; A NUMBER OF STEPS IS THE LENGTH OF THE COUNTEREXAMPLE.

Model, switches	Result	States
Close, second set omits inherited	Thm. 1 fails (19 steps); two fast certificates (23)	–
Close, all rules	no violation	0.65 M
Overlap, all rules	no violation	5.2 M
Overlap, no release	same-verdict fails (15)	–
Overlap, late votes not evidence	next close cannot verify a correct report (14)	–
Overlap, late votes in notarization	a correct report becomes unusable (9); safety holds	5.5 M
Handoff, early votes on fast path, same or disjoint K	Thm. 1 fails for S_2 (22, 23)	–
Handoff, rule, same / disjoint / 3 shared	no violation	8.3 / 3.7 / 121 M
Prefix N=4, prefix guard, all overlaps	no violation	0.3–4.0 M
Prefix N=4, target only, 2 shared	conflicting finality (13)	–
Prefix N=4, child inherits witness, not same K	conflicting finality (12–14)	–
Prefix N=6, target only or inherited witness	conflicting finality (simulation)	–
Recovery N=4, tip always extensible, all K	conflicting finality (16)	–
Recovery N=4, re-notarization, all K	no violation	0.7–3.9 M
Close, Overlap, Handoff, Recovery: Rule 2 keeps a side	no violation	0.3–5.9 M
Resolve N=4, no termination rules	fork open after two good epochs (sim.)	–
Resolve N=4, termination rules, epochs 2–3 good	no violation	4.6 M
Recover, all first votes, bottom-up	both sides final (40)	–
Recover, settled votes, bottom-up, owner forks	Thm. 1 fails (15)	–
Recover, recipe of Section VI	no violation to 17 steps; sim.	21 M

X. Implementation and Evaluation

The evaluation asks three questions. Does RAI resolve the forks Nano leaves open, and what does that cost in confirmation rate and latency? Does it keep confirming payments while an epoch closes, where a stop-and-drain transition stops, also when a validator is Byzantine? Does a committee change disturb it?

Prototype. We implemented RAI in RsNano [9], a Rust implementation of the Nano node [5]. Account elections follow Section V, with late notarization, the early-vote rule, and the prefix-wide overlap rule; checkpoints carry origin-stamped locks and are built by the recipe of Section VI. Epochs end at fixed times $T_0 + kD$ on every validator's local clock. Every vote, frozen report, decided checkpoint, and overlap certificate is synced to disk before a signature that depends on it leaves the process, so a restarted validator refuses whatever its earlier signatures forbid. Set reconstruction uses rateless invertible Bloom lookup tables [7]. Agreement is one leader-based election per handover in the style of Kudzu [8]: a validator votes for a proposal only after reconstructing its

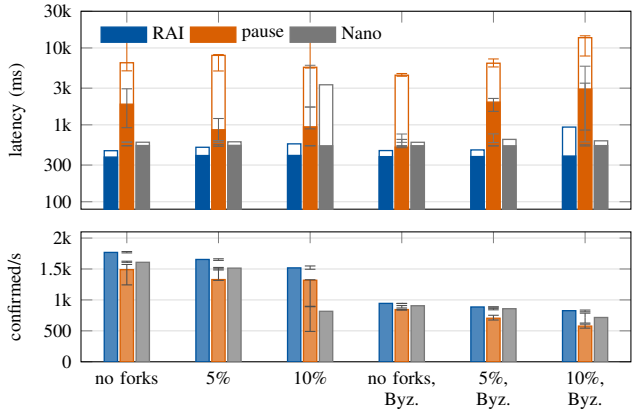


Fig. 4. RAI ($f = p = 1$), pause, and Nano across six regions: 2,000 blocks/s offered without a Byzantine validator, 1,000 blocks/s with one, and 5% or 10% forks. Top: median (filled) and 95th-percentile (outline) latency of non-fork blocks. Bottom: non-fork blocks confirmed per second over the run. Bars are medians over three runs, whiskers their range.

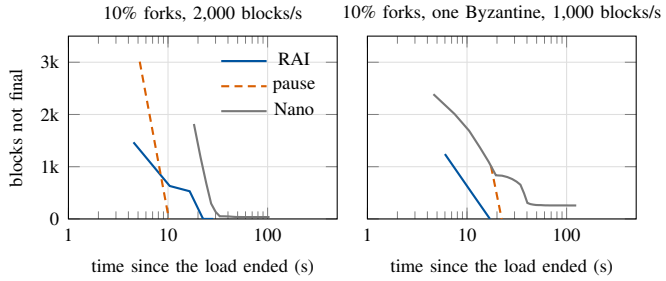


Fig. 5. Blocks not yet final on the validator with the most of them, after the load ended: forked positions and the blocks behind them (one run each, sampled from the first check after the load). RAI and pause finalize all of them, identically on every validator; Nano leaves blocks open.

reports, running the recipe, and matching the state digest; leaders rotate, and a round whose leader stays silent times out after 2 s. A validator does not repeat a request for votes before its reply could arrive: the floor is set to the longest measured round trip. The supplement lists where the prototype departs from the protocol.

What we compare. *RAI* is the protocol of this paper with $f = p = 1$ on six members: a certificate needs four of six, and the fast path five settled first votes. *Pause* is the same code with one switch flipped: no instance of epoch $e + 1$ starts before S_e is decided locally. It is the stop-and-drain transition RAI set out to avoid, with reports, recipe, agreement, and installation unchanged. *Nano* is the RsNano `develop` node (commit `ff2b32710`) with one optimization of RAI's prototype ported to it, checking a fork block's signature once: a block is confirmed once final votes reach 67% of the online weight, five of six here, after a round of non-final votes. It has no epochs and no committee, and since final votes cannot be withdrawn, a fork whose votes split leaves neither side able to confirm. RAI and Nano both tolerate one Byzantine validator, but RAI's normal path needs fewer votes.

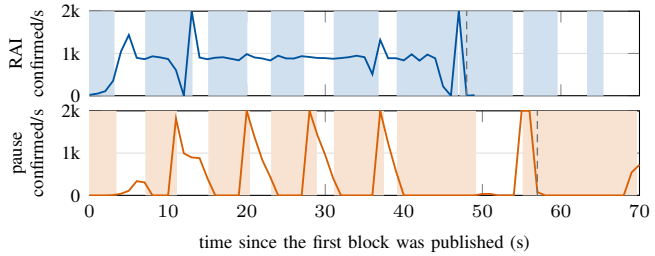


Fig. 6. Non-fork blocks confirmed per second at 1,000 blocks/s offered, with 10% forks and one Byzantine validator, across six regions (one run each). Shaded: an epoch is closing, from the first validator leaving it to the last one seeing its checkpoint decided; dashed: the end of the load. Pause stops confirming during each close, then catches up in bursts (cut off at 2,000/s); RAI keeps confirming through the closes.

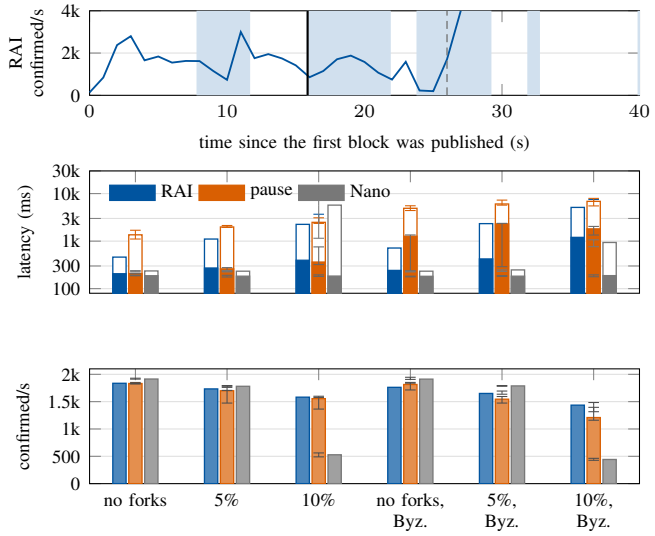


Fig. 7. A committee change under load, on one host at 2,000 blocks/s: four seats ($f = 1$, $p = 0$), two members hand their weight to standby validators in epoch 0, so the committee of epoch 2 replaces them. Top: RAI with 10% forks, the solid line marking the first validator entering epoch 2 (one run). Below: latency and confirmation rate as in Figure 4, three runs each, without and with a Byzantine member that keeps its seat.

Setup. Six equal-weight validators run on separate Google Cloud VMs (n2-standard-4, four vCPUs), one each in Warsaw, Iowa, Montréal, Las Vegas, Taiwan, and Melbourne, with round trips from about 30 to over 300 ms. A load generator on a seventh VM in Warsaw funds 45,000 accounts and then publishes 45,000 blocks, each on the frontier of a randomly chosen account, with the same random choices for the three systems. It offers 2,000 blocks/s without a Byzantine validator and 1,000 blocks/s with one, the highest rate at which we saw RAI settle with an equivocating validator across these links. In the fork scenarios it publishes a conflicting sibling for 5% or 10% of the blocks, each sibling to half of the validators, and never builds on a forked position, so what is left open measures what the validators resolve on their own. In the Byzantine scenarios one validator runs a modified node: it votes for both blocks of every fork, signs two versions of its report and sends

each to half of the validators, and leads the first round of every close with two different proposals. Nano's Byzantine validator votes for both blocks of every fork, the one attack Nano has a phase for. RAI and pause use 8 s epochs, so a close starts about every 8 s during the load. The *confirmation rate* counts the non-fork blocks the client sees confirmed per second over the whole run, so it stays below the offered load. *Latency* runs from a block's first publication to its confirmation at the Warsaw validator and counts non-fork blocks only. After the load, the validators keep running until no block is open on any of them, or until none has become final for 45 s or the harness's time limit is reached. Each cell ran three times; we report the median and the range.

Latency and confirmation rate (Figure 4). Without forks RAI has a median of 381 ms and a 95th percentile of 461 ms against Nano's 538 and 594 ms, and confirms 1,768 against 1,608 blocks/s. Nano needs five of six votes in both of its rounds, so it waits for a distant validator twice; RAI's normal path waits for four, and 27 to 65% of its finalizations, depending on the validator and run, take the one-round fast path. With forks RAI's median stays at 399 and 400 ms and its 95th percentile at 511 and 566 ms, at 1,654 and 1,518 blocks/s. Nano keeps its median (541 and 534 ms), but with 10% forks its 95th percentile rises to 3.3 s and it confirms 816 blocks/s (491 to 897). With a Byzantine validator at 1,000 blocks/s, RAI's median is 386 to 393 ms and its 95th percentile 463 to 935 ms, at 826 to 944 blocks/s; Nano's are 534 to 537 ms and 593 to 647 ms, at 716 to 906 blocks/s. Pause is far behind both: its median is 0.5 to 2.9 s and its 95th percentile 4.4 to 13.6 s, ten to sixteen times RAI's in every cell, and RAI confirms 12 to 43% more blocks per second.

RAI resolves forks (Figure 5). With forks, RAI finalized every open block within 20 to 27 s of the end of the load without a Byzantine validator and within 17 to 29 s with one; pause took 10 to 24 s and 10 to 34 s. In all of these runs every validator cemented the same state. RAI is often the slower of the two here; one reason is that its recipe counts settled first votes only, so a fork whose first votes all came before the previous checkpoint was installed waits one more epoch. Nano did not finalize them all. When the harness stopped waiting it had 36 to 1,020 blocks open on a validator without a Byzantine validator and 19 to 1,174 with one. Its validators held the same blocks in all 12 runs with forks, but in 6 of them had cemented different numbers of them.

RAI keeps confirming while an epoch closes (Figure 6). Pause holds new blocks while an epoch closes and then confirms them in a burst. With 10% forks and a Byzantine validator it had 22 to 29 seconds without a single confirmation during the 45 s load, up to 10 in a row, and averaged 442 to 549 blocks/s while an epoch closed. RAI, on the same load, had one or two such seconds, never two in a row, and averaged 785 to 892 blocks/s while an epoch closed. Without a Byzantine validator RAI had no such second and averaged 1,522 to 1,962 blocks/s during closes at 2,000 offered; pause had 7 to 10 and averaged 162 to 967. RAI's closes are not short: with 10% forks and a Byzantine validator the longest of each run takes 7.6 to 8.0 s, about the length of an epoch, since a close must wait for the reports of all five honest validators (pause: 14.5 to 18.9 s). But no block waits for them. Each validator finds 14,000 to 25,000 of the 45,000 blocks eligible for the overlap rule.

A committee change (Figure 7). These runs are on one host: six processes on a MacBook Air (Apple M1, eight cores) over loopback, 48,000 blocks at 2,000 blocks/s, so the delays are dominated by CPU contention and the numbers do not compare with the cells above. With four seats ($f = 1$, $p = 0$), two members hand their weight to two standby validators in epoch 0, so the committee derived from epoch 0's state, the one epoch 2 runs, replaces both. RAI keeps confirming through the handover. Across the three scenarios it confirms 1,582 to 1,836 blocks/s with medians of 206 to 396 ms, and resolves every fork. Pause's 95th percentile is 2.9, 1.8, and 1.1 times RAI's without forks, with 5%, and with 10% forks; Nano leaves forks open (687 to 1,135 blocks with 5%) and confirms 528 blocks/s with 10%. With a Byzantine member that keeps its seat through the change, RAI again resolves every fork; its median is 242 to 1,200 ms against pause's 1.3 to 2.3 s and its 95th percentile 0.7 to 5.1 s against 4.9 to 6.9 s. RAI confirms slightly less than pause without forks (1,762 against 1,814 blocks/s) and more with 5% and 10% forks (1,650 and 1,436 against 1,545 and 1,209).

Restarts and handoff. A second harness, on the same host, kills validators mid-load or as a close starts, restarts them on their data, and checks every logged signature for conflicts. Restarted validators rejoined by checkpoint catch-up, and a departing member killed when its last close starts and kept down 30 s left every checkpoint decided identically. No run produced a conflicting signature. Synced signing costs about 130 ms at the median and nothing in confirmation rate. The supplement reports these runs.

Threats to validity. RAI's normal path needs four votes and Nano's five, so part of RAI's lower latency comes from the smaller quorum, not from the protocol's other rules. Latency is measured at one validator, beside the load generator; a client elsewhere would see other numbers. The cells with a Byzantine validator run at half the rate of the others and do not compare with them. Across these links RAI did not settle with an equivocating validator at 1,500 blocks/s with 10% forks or at 2,000 with 5%: equivocation keeps thousands of forks contested until a checkpoint decides them, the evidence traffic overflows the queues of the slowest validator, and each close waits for that validator's report. The committee change and the restarts were not run across regions. Three runs per cell bound the spread only roughly, and process kills test crash consistency, not power loss.

XI. Related Work

Guerraoui et al. characterize the consensus number of asset transfer [1]. FastPay settles prefunded payments without a global order [2]; Astro builds asynchronous payments on broadcast and, in its extended version, discusses replica

reconfiguration with view-tagged messages and account-log transfer [3], [10]; Zef adds private coins [4]; Nano uses account chains with separate send and receive blocks and open representative voting [5]. RAI's difference is the frozen-report evidence that preserves first- and final-vote obligations across a boundary while the next epoch is already voting, and the locks that give a contested position a decision, not a deadlock.

Sui Lutris uses consensus for shared objects and checkpoint sequencing, including end-of-epoch handling of processed certificates [11]; it sequences certificates continuously, so it has a stream to drain, where RAI has none. Cuttlefish broadens the fast path with collective objects and FastUnlock [12]; RAI's recovery of a locked account is narrower and needs no new primitive. Pastro studies permissionless asynchronous asset transfer under a dynamic adversary [13]; dynamic Byzantine reliable broadcast and asynchronous Byzantine reconfiguration address broader changing-participation settings [10], [14]. PDCC combines certificate-based payments with membership change and periodic settlement; its installation rule accepts certificates for the installed configuration and assumes correct members common to consecutive configurations [15], whereas RAI preserves old-epoch certificates assembled later and needs no members in common.

Agreement with external validity is a standard interface [6]; we instantiate it with a leader-based election in the style of Simplex and Kudzu [16], [8]. Authenticated set reconciliation is mature [17], [18], [7]. Model checking small configurations of Byzantine protocols with TLC is common practice [19]; here it served as a regression test for the handover rules.

XII. Limitations and Conclusion

RAI's checkpoints grow with the unresolved forks they keep, and locks, late supports, and overlap witnesses must be kept while a lock they can release survives; we give no garbage-collection policy beyond the prototype's four-epoch cap. The committee model is authenticated and equal-weight with an externally kept fault bound. With an equivocating validator and 10% forks the prototype settles across six regions at 1,000 blocks/s but not at 1,500; the committee change and the restarts were measured on one host only; each cell ran three times; and process kills test crash consistency, not power loss. The models cover at most two positions and four epochs and check the recipe only by simulation and to 17 steps.

Within these limits RAI does what it set out to do. It keeps account-local payments safe across committee changes without stopping: the next epoch votes while the previous one closes, immutable reports preserve whatever the old epoch could still confirm, and four overlap rules keep the two epochs from contradicting each other. It keeps at most one side of every contested position, identically everywhere, and finalizes that side within two epochs of normal operation, so the owner always gets the account back. Across six regions, against a stop-and-drain transition built from the same code, it keeps confirming through every boundary, with a 95th percentile ten to sixteen times lower. Against a system without epochs it finalizes the forks that system leaves open, with a lower median and a lower 95th percentile in most cases, on a normal path that needs fewer votes, and it confirms almost twice as many blocks per second when forks are frequent. In return it can change its validators, and its protocol: every validator agrees on the epoch boundaries and the checkpoint that closes each epoch, so a protocol change can take effect at an agreed epoch, where Nano appends an epoch block to every account.

AI Tool Usage

ChatGPT assisted with structuring the manuscript, with source checking, protocol review, and proof exposition, and helped write the TLA$^+$ models, which were checked with TLC. Claude Code (Anthropic) assisted with the prototype implementation, the benchmark harness and the stop-and-drain switch, ran the experiments of Section X and produced their figures, diagnosed defects in the prototype, and drafted the text and figures, which the authors revised. The proof arguments were reviewed by the authors and have not been machine checked beyond the finite configurations of Section IX. No data in this paper is AI-generated: all measurements come from the prototype runs described in Section X.

References

[1] R. Guerraoui, P. Kuznetsov, M. Monti, M. Pavlovic, and D.-A. Seredinschi, "The consensus number of a cryptocurrency," in *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 307–316.

[2] M. Baudet, G. Danezis, and A. Sonnino, "FastPay: High-performance byzantine fault tolerant settlement," in *Proceedings of the ACM Conference on Advances in Financial Technologies (AFT)*, 2020.

[3] D. Collins, R. Guerraoui, J. Komatovic, M. Monti, A. Xygkis, M. Pavlovic, P. Kuznetsov, Y.-A. Pignolet, D.-A. Seredinschi, and A. Tonkikh, "Online payments by merely broadcasting messages (extended version)," 2020, extended version of the DSN 2020 paper; Appendix A describes asynchronous reconfiguration.

[4] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, "Zef: Low-latency, scalable, private payments," in *Proceedings of the Workshop on Privacy in the Electronic Society (WPES)*, 2023, pp. 1–16.

[5] C. LeMahieu, "Nano: A feeless distributed cryptocurrency network," White paper. [Online]. Available: https://content.nano.org/whitepaper/Nano_Whitepaper_en.pdf

[6] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and efficient asynchronous broadcast protocols," in *Advances in Cryptology – CRYPTO 2001*, ser. LNCS, vol. 2139. Springer, 2001, pp. 524–541.

[7] L. Yang, Y. Gilad, and M. Alizadeh, "Practical rateless set reconciliation," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 595–612.

[8] V. Shoup, J. Sliwinski, and Y. Vonlanthen, "Kudzu: Fast and simple high-throughput BFT," in *39th International Symposium on Distributed Computing (DISC 2025)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 356. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 42:1–42:19.

[9] RsNano contributors, "Rsnano software repository," Software repository, base of the RAI prototype and of the evaluated baseline. [Online]. Available: https://github.com/rsnano-node/rsnano-node

[10] R. Guerraoui, J. Komatovic, P. Kuznetsov, Y.-A. Pignolet, D.-A. Seredinschi, and A. Tonkikh, "Dynamic byzantine reliable broadcast," 2020, extended technical report, version 2.

[11] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, "Sui lutris: A blockchain combining broadcast and consensus," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024, pp. 2606–2620.

[12] L. Kokoris-Kogias, A. Sonnino, and G. Danezis, "Cuttlefish: Expressive fast path blockchains with FastUnlock," 2023, version 1.

[13] P. Kuznetsov, Y.-A. Pignolet, P. Ponomarev, and A. Tonkikh, "Permissionless and asynchronous asset transfer," in *35th International Symposium on Distributed Computing (DISC 2021)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 209. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 28:1–28:19.

[14] P. Kuznetsov and A. Tonkikh, "Asynchronous reconfiguration with byzantine failures," *Distributed Computing*, vol. 35, pp. 477–502, 2022.

[15] L. Shangguan, "Transaction-driven dynamic reconfiguration for certificate-based payment systems," 2026, initial preprint, version 1, January 14, 2026.

[16] V. Shoup, "Sing a song of simplex," in *38th International Symposium on Distributed Computing (DISC 2024)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 319. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 37:1–37:22.

[17] M. T. Goodrich and M. Mitzenmacher, "Invertible bloom lookup tables," in *49th Annual Allerton Conference on Communication, Control, and Computing*, 2011, pp. 792–799.

[18] D. Eppstein, M. T. Goodrich, F. Uyeda, and G. Varghese, "What's the difference? efficient set reconciliation without prior context," in *Proceedings of the ACM SIGCOMM 2011 Conference*, 2011, pp. 218–229.

[19] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.